

Individual Assessment Coversheet

To be attached to the front of the assessment.

Campus: Tyervalley
 Faculty: BSc IT
 Module Code: ITECA3-12
 Group: 2
 Lecturer's Name: Mr. J Kamwaya
 Student Full Name: Colin Swart
 Student Number: EDUV4861087

Indicate	Yes	No
Plagiarism report attached		x

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment, or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the institution's policy and regulations on honesty in academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature <i>CSwart</i>	Date 25-02-2026
----------------------------	--------------------

Lecturer's Comments:

Marks Awarded:	%
----------------	---

Signature	Date
-----------	------

ITECA3-12 Initial Project

Deliverable 2 – Design & Prototype

Hosted at: <https://vossiehallphp75163.azurewebsites.net>

Contents

2.1 Introduction	3
2.2 Prototyping.....	4
2.2a Main Website Prototype.....	4
Login Page.....	4
Registration Page	5
Browse Listings / Home Feed	6
Listing Detail Page	7
Create Listing Form	8
QR Transaction Pages.....	9
2.2b Admin Website Prototype	10
Admin Dashboard	10
2.3 Designing.....	11
2.3a Class Responsibility Collaborator (CRC) Cards	11
2.3b Enhanced Entity Relationship Diagram (EERD).....	12
2.3c Context Diagram.....	12
2.3d Data Flow Diagram (DFD)	13
2.3e Use Case Diagram	14
2.3f Database Design (Schema).....	15
USERS	15
LISTINGS	15
TRANSACTIONS.....	15
2.4 Coding	16
2.4a Platform Screenshots:	16
2.4b Sample PHP Code	16
2.4c Sample HTML Code	17
2.4d Sample JavaScript Code	17
2.4e Sample CSS Code	18
2.4f Sample MySQL Table Screenshots.....	19
2.5 Conclusion	21

2.1 Introduction

Vossie Trading Hall is a student-only marketplace where Eduvos students can buy and sell goods directly with each other. Users register with institutional @vossie.net email addresses which keeps the platform exclusive and limited to verified students in Eduvos.

The platform is built with HTML, CSS, JavaScript, PHP, and MySQL, and is responsive across phones, tablets, and desktops. The frontend uses Tailwind CSS with custom styling, and no CMS tools are used. The solution is hosted live on Microsoft Azure.

In addition to the core marketplace features, the system includes an admin dashboard. Admin users can manage accounts, assign or remove admin access, and moderate listings when needed. Regular student users can browse listings, create their own listings, and confirm completed transactions using QR-code-based confirmation. This keeps buyer and seller actions distinct while still connected through a controlled transaction flow.

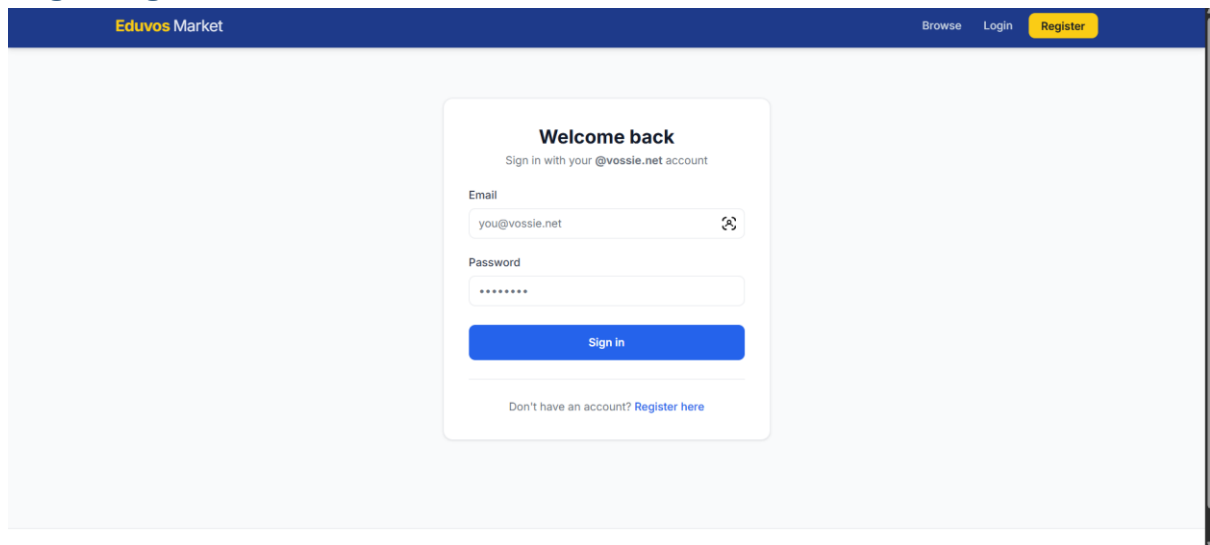
This deliverable presents the live interface, system diagrams, database design, and selected implementation code that powers the platform.

2.2 Prototyping

The prototype screens are fully responsive and have been tested at two breakpoints: smartphone ($\leq 576\text{px}$), and desktop ($\geq 993\text{px}$).

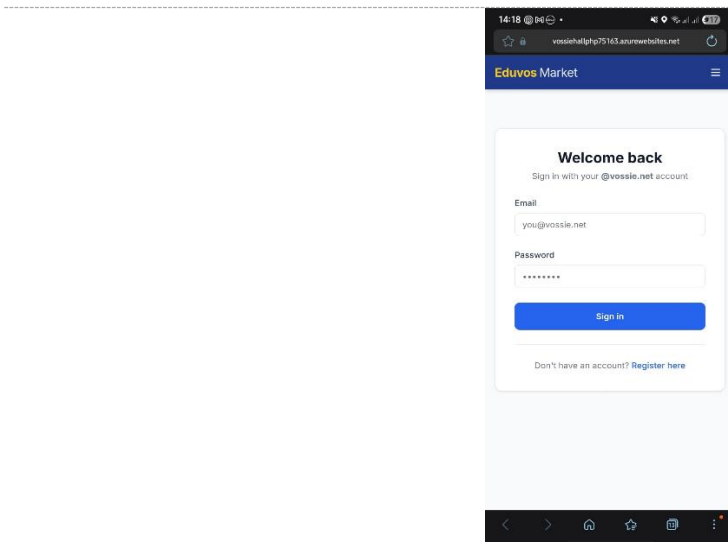
2.2a Main Website Prototype

Login Page



The desktop view of the login page features a dark blue header with the 'Eduvos Market' logo on the left and 'Browse', 'Login', and a yellow 'Register' button on the right. The main content area is light gray and contains a white login card. The card has the heading 'Welcome back' and the instruction 'Sign in with your @vossie.net account'. It includes an 'Email' field with the placeholder 'you@vossie.net', a 'Password' field with masked characters, a blue 'Sign in' button, and a link 'Don't have an account? Register here'.

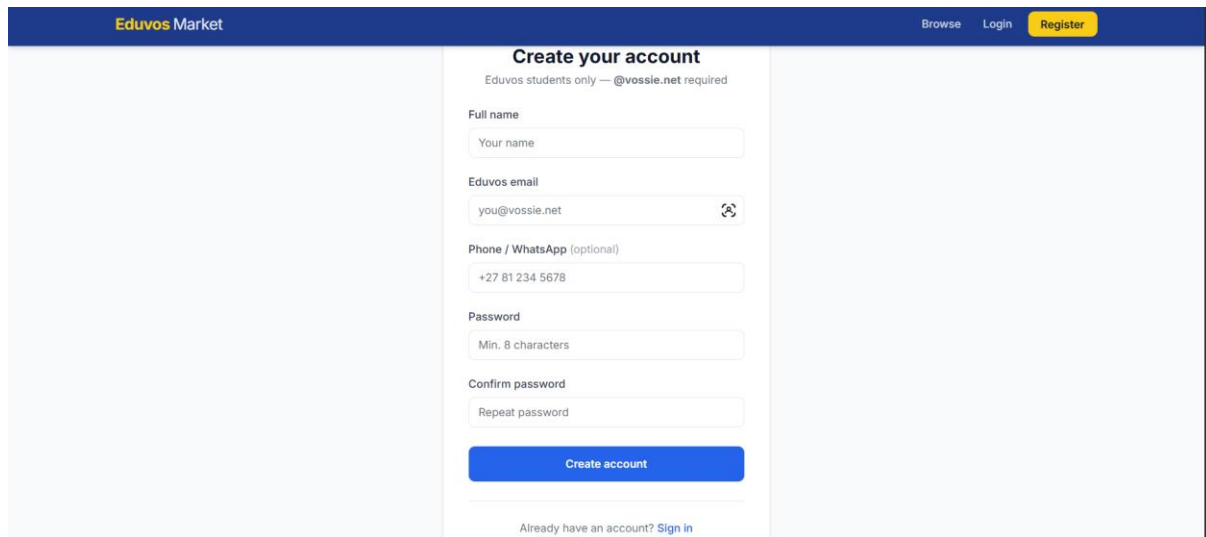
Figure 1: Login page – Desktop view



The mobile view of the login page is shown within a smartphone frame. It maintains the same layout as the desktop version but is scaled to fit the smaller screen. The 'Eduvos Market' header and the login card with its fields and buttons are clearly visible and centered.

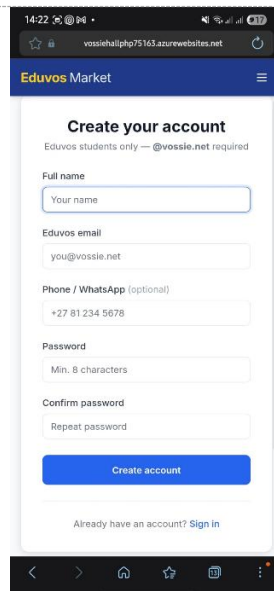
Figure 2: Login page – Mobile view

Registration Page



The desktop view of the registration page features a dark blue header with the 'Eduvos Market' logo on the left and 'Browse', 'Login', and a yellow 'Register' button on the right. The main content area is white and contains the 'Create your account' form. The form includes a sub-header 'Eduvos students only — @vossie.net required' and several input fields: 'Full name' (placeholder: 'Your name'), 'Eduvos email' (placeholder: 'you@vossie.net' with a copy icon), 'Phone / WhatsApp (optional)' (placeholder: '+27 81 234 5678'), 'Password' (placeholder: 'Min. 8 characters'), and 'Confirm password' (placeholder: 'Repeat password'). A blue 'Create account' button is positioned below the form, followed by a link 'Already have an account? Sign in'.

Figure 3: Registration page – Desktop view



The mobile view of the registration page is shown within a browser window. The header is a dark blue bar with the 'Eduvos Market' logo and a hamburger menu icon. The form, titled 'Create your account', includes the same sub-header and input fields as the desktop version: 'Full name', 'Eduvos email', 'Phone / WhatsApp (optional)', 'Password', and 'Confirm password'. A blue 'Create account' button and a 'Sign in' link are at the bottom of the form. The browser's address bar shows the URL 'vossiehalp75163.azurewebsites.net', and the mobile status bar at the bottom displays the time '14:22' and various icons.

Figure 4: Registration page – Mobile view

Browse Listings / Home Feed

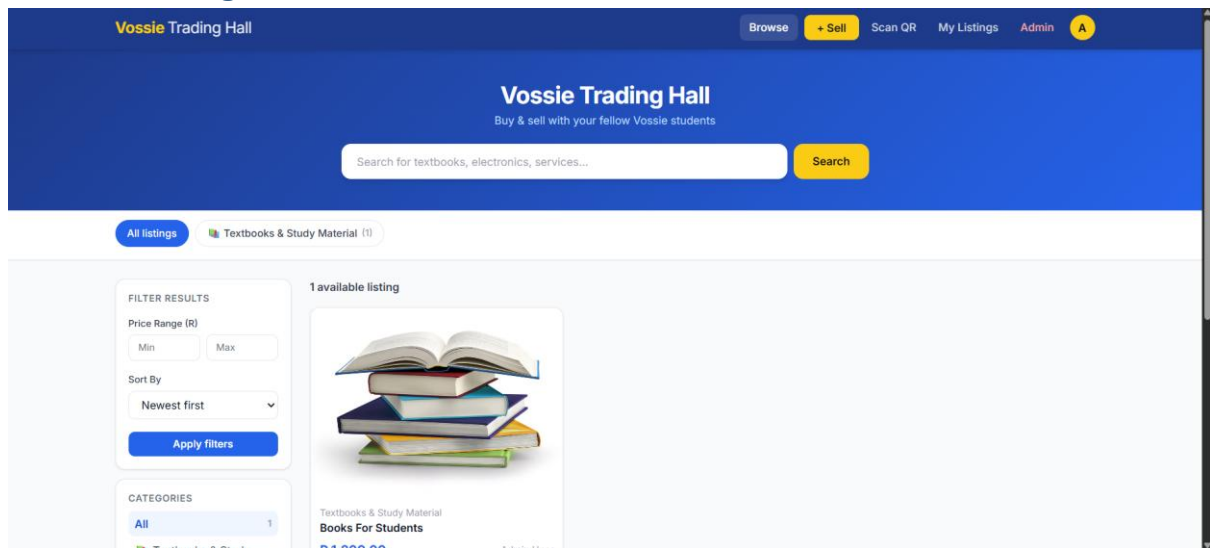


Figure 5: Browse listings – Desktop view

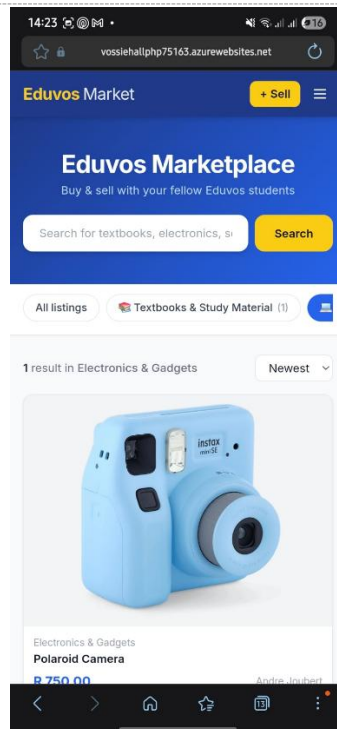


Figure 6: Browse listings – Mobile view

Listing Detail Page

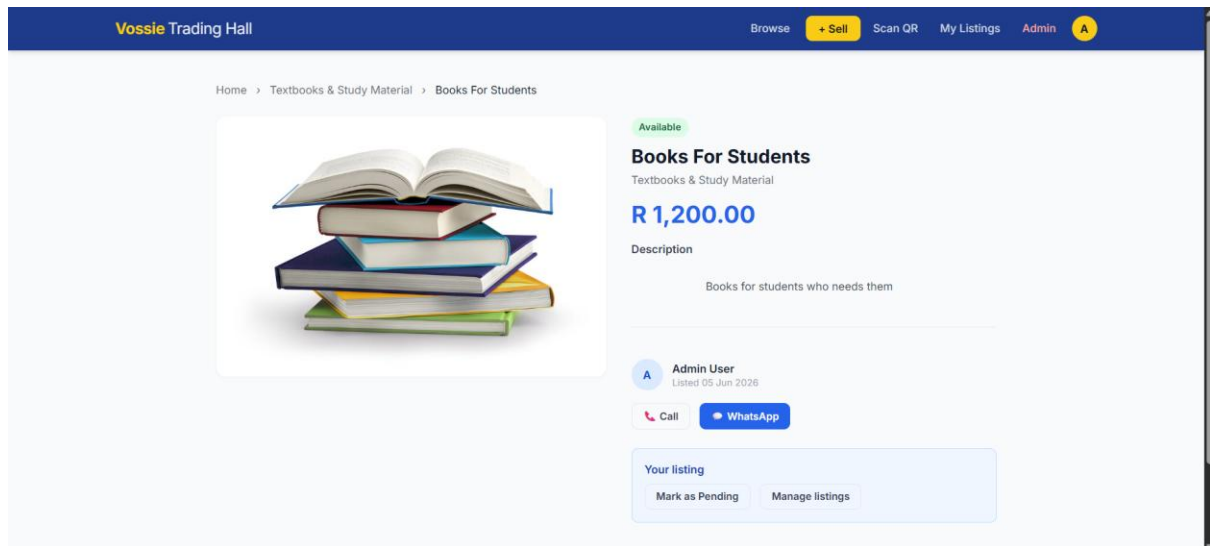


Figure 7: Listing detail – Desktop view

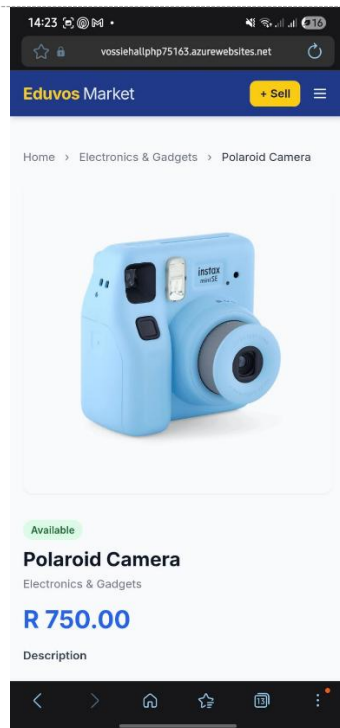


Figure 8: Listing detail – Mobile view

Create Listing Form

Vossie Trading Hall

Browse+ SellScan QRMy ListingsAdminA

Post a listing

Title *
e.g. Introduction to Economics textbook

Description
Condition, edition, any details...

Category *
Select a category...

Listing type *
☒ I'm selling at a price ☐ I'm looking to buy (Wanted)

Price (R) *
0.00

Contact number / WhatsApp

Figure 9: Create listing form – Desktop view

QR Transaction Pages

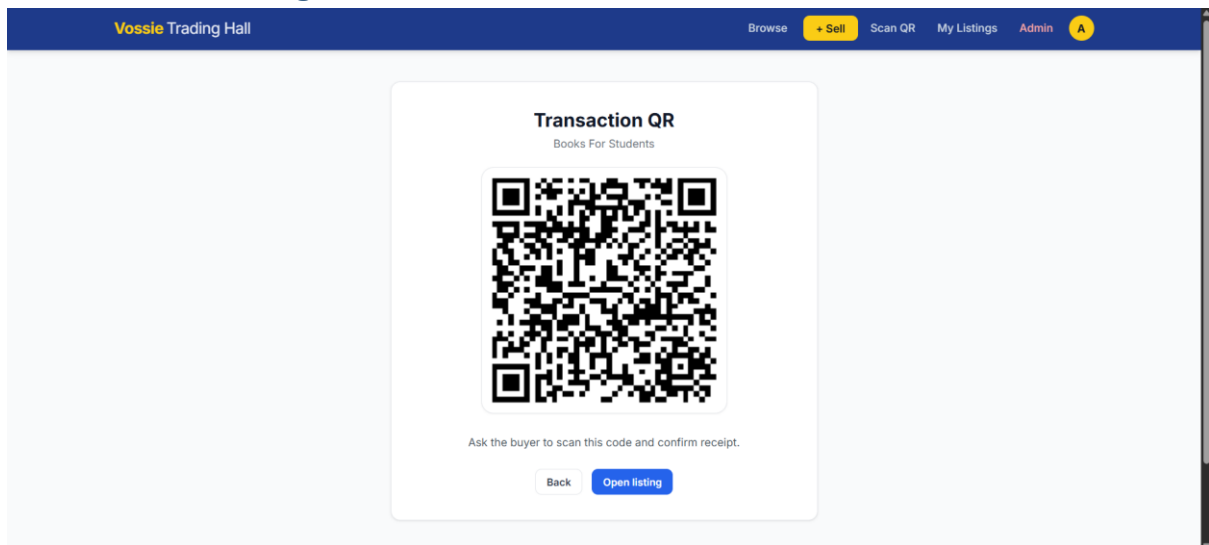


Figure 10: QR token display – Seller view

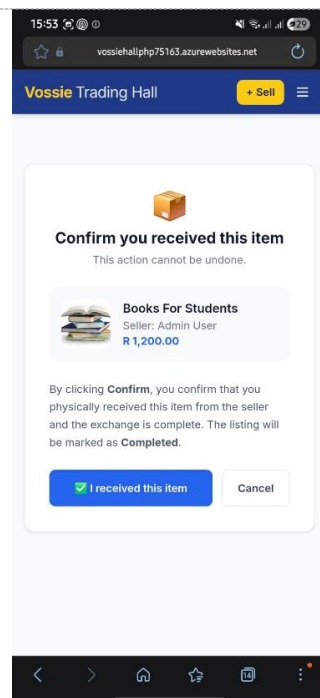


Figure 11: QR scan confirmation – Buyer view

2.2b Admin Website Prototype

Admin Dashboard

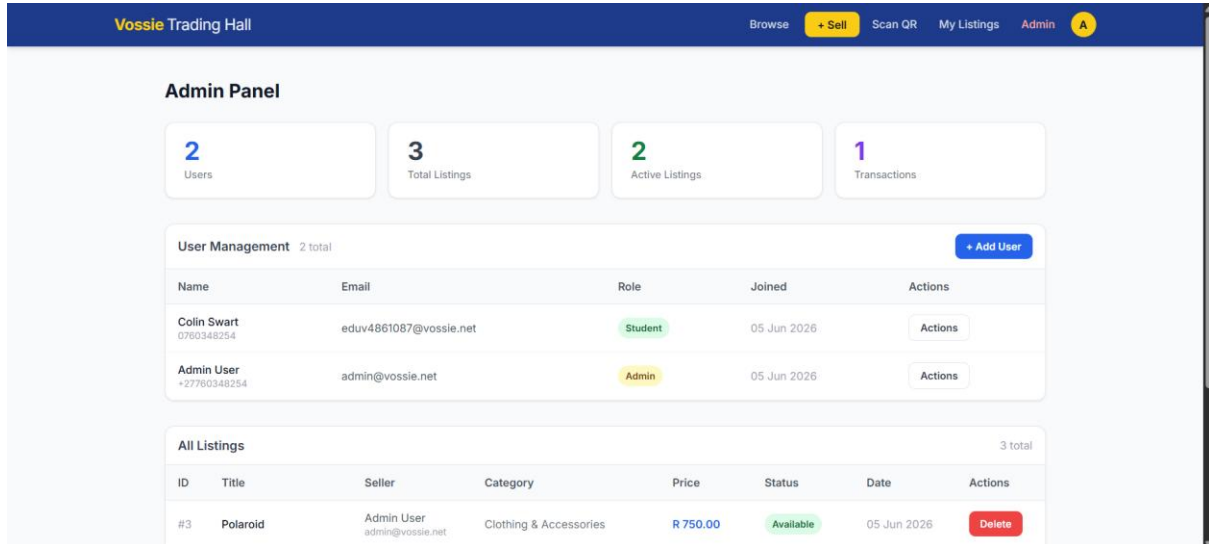


Figure 12: Admin dashboard – Desktop view

2.3 Designing

2.3a Class Responsibility Collaborator (CRC) Cards

The following CRC cards describe the key classes in the Vossie Trading Hall system, their responsibilities, and the other classes they collaborate with.

Class	Responsibilities	Collaborators
User	Register and login; manage own profile; create and browse listings; initiate transactions	AuthService, Listing, Transaction, Role
Role	Define permission sets for each user type (Admin, StudentUser); restrict or grant access	User, AdminController, AuthService
AuthService	Validate @vossie.net credentials; enforce session tokens; check role permissions on each request	User, Role
Listing	Store product title, description, price, category, images, and status lifecycle (active/sold/removed)	User, Transaction, SearchService
ListingImage	Store image URLs associated with a listing; support multiple images per listing	Listing
Transaction	Record a confirmed buyer-seller exchange; hold QR token and completion status	Listing, User, QRService
QRService	Generate a unique QR token per transaction; validate the token when the buyer scans	Transaction, Listing
AdminController	Perform CRUD on users and role assignments; moderate listings; access reporting data	User, Role, Listing, Transaction
SearchService	Filter and rank listings by category, price range, keyword, and recency	Listing

2.3b Enhanced Entity Relationship Diagram (EERD)

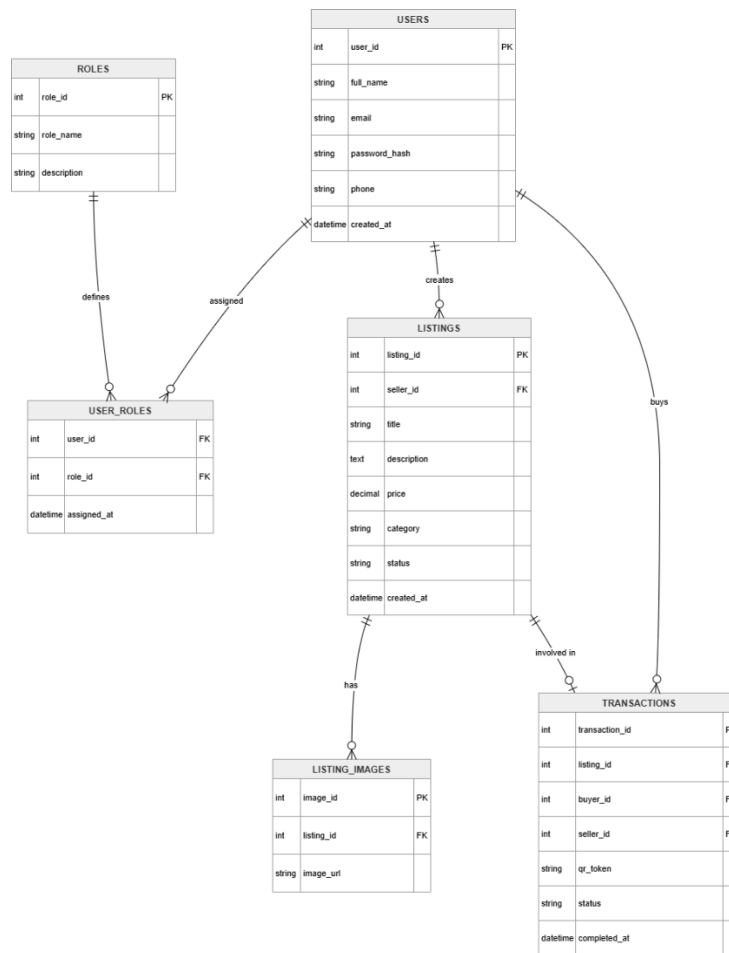


Figure 13: Enhanced Entity Relationship Diagram – Vossie Trading Hall

2.3c Context Diagram

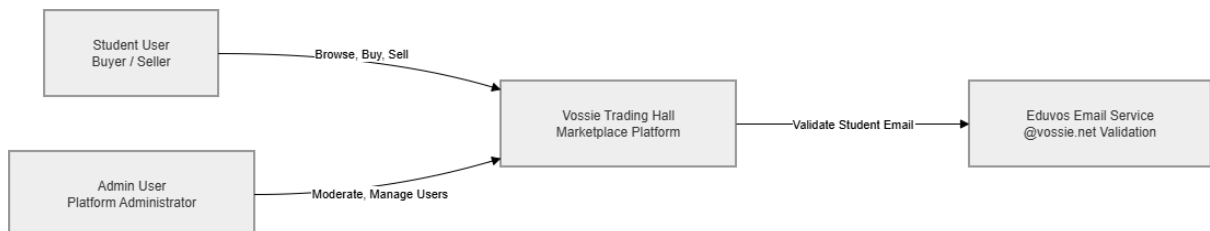


Figure 14: Context Diagram – Vossie Trading Hall

2.3d Data Flow Diagram (DFD)

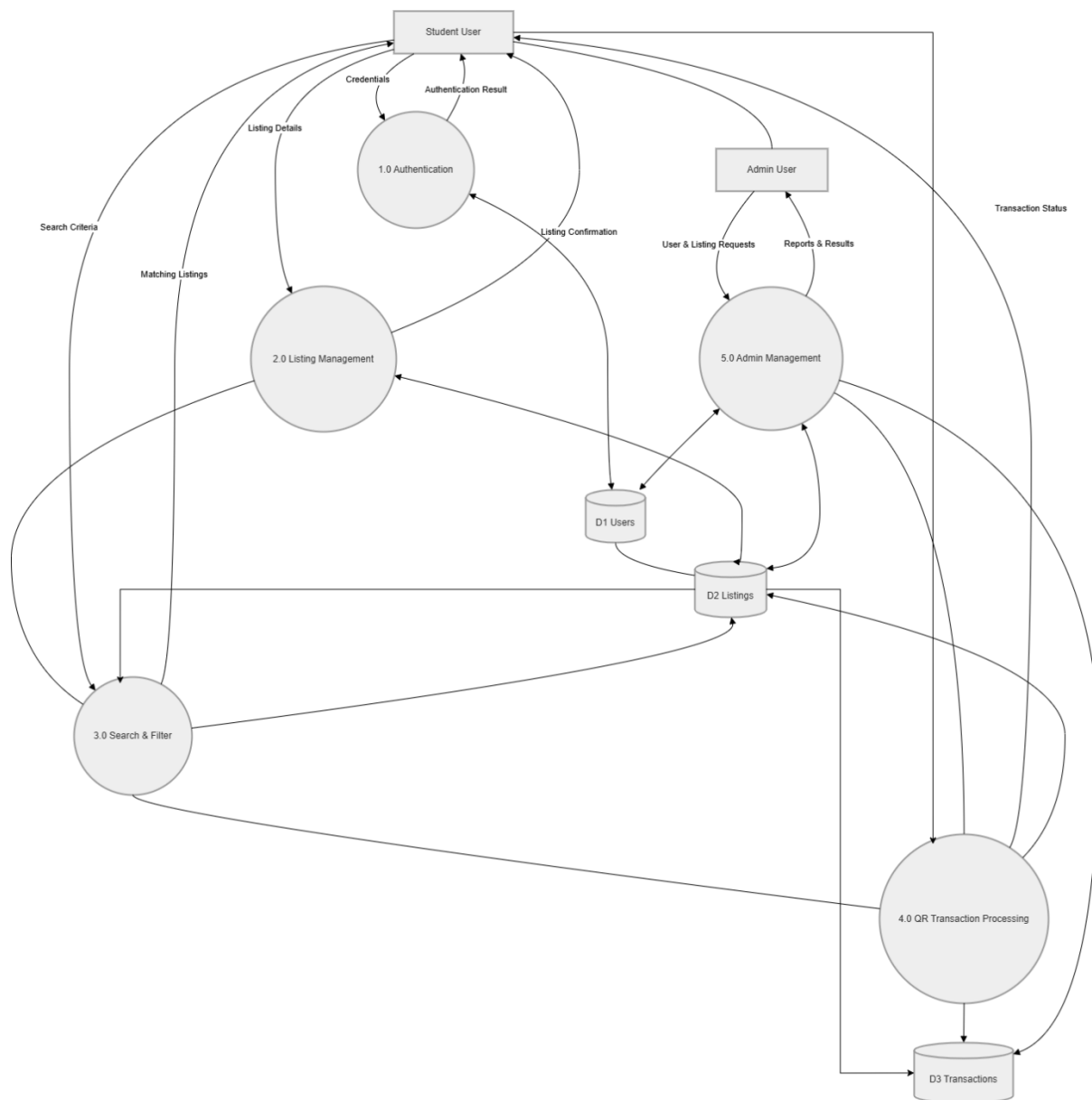


Figure 15: Data Flow Diagram Level 1 – Vossie Trading Hall

2.3e Use Case Diagram

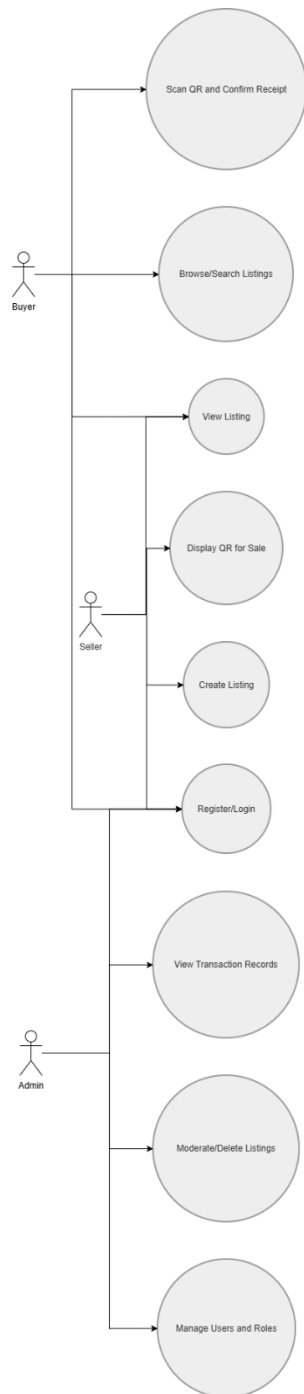


Diagram 15: Use case Diagram

2.3f Database Design (Schema)

The database consists of six relational tables. All tables use InnoDB for foreign key enforcement. The schema below defines the column names, data types, and constraints for each table.

USERS

Column	Type	Constraints
id	INT AUTO_INCREMENT	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
email	VARCHAR(150)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL
phone	VARCHAR(20)	NULL
is_admin	TINYINT(1)	DEFAULT 0
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

LISTINGS

Column	Type	Constraints
id	INT AUTO_INCREMENT	PRIMARY KEY
user_id	INT	FK → users(id), NOT NULL
title	VARCHAR(200)	NOT NULL
description	TEXT	NULL
price	DECIMAL(10,2)	NOT NULL
price_type	ENUM('fixed','wanted')	DEFAULT 'fixed'
category	VARCHAR(60)	NOT NULL
image	VARCHAR(255)	NULL
phone	VARCHAR(20)	NULL
status	ENUM('available','pending','completed')	DEFAULT 'available'
qr_token	CHAR(64)	NULL
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

TRANSACTIONS

Column	Type	Constraints
id	INT AUTO_INCREMENT	PRIMARY KEY
listing_id	INT	FK → listings(id), NOT NULL
seller_id	INT	FK → users(id), NOT NULL
buyer_id	INT	FK → users(id), NOT NULL
qr_token	CHAR(64)	NOT NULL

confirmed_at	DATETIME	NULL
---------------------	----------	------

2.4 Coding

2.4a Platform Screenshots:

The platform screenshots from question 2.2 Are the platform which is currently hosted screenshots aswell

2.4b Sample PHP Code

The following PHP excerpt handles user authentication which validates that the submitted email belongs to the @vossie.net domain and that the hashed password matches the stored hash. On success, a session is started and the user's role is loaded for RBAC enforcement throughout the application.

```
<?php
// login.php – Authentication handler
session_start();
require_once 'db.php';

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $email = trim($_POST['email']);
    $password = $_POST['password'];

    // Enforce @vossie.net domain
    if (!str_ends_with($email, '@vossie.net')) {
        $error = 'Only @vossie.net accounts are permitted.';
    } else {
        $stmt = $pdo->prepare(
            'SELECT u.user_id, u.full_name, u.password_hash,
             GROUP_CONCAT(r.role_name) AS roles
            FROM users u
            LEFT JOIN user_roles ur ON u.user_id = ur.user_id
            LEFT JOIN roles r ON ur.role_id = r.role_id
            WHERE u.email = ? GROUP BY u.user_id'
        );
        $stmt->execute([$email]);
        $user = $stmt->fetch();

        if ($user && password_verify($password, $user['password_hash'])) {
            $_SESSION['user_id'] = $user['user_id'];
            $_SESSION['full_name'] = $user['full_name'];
            $_SESSION['roles'] = explode(',', $user['roles'] ?? '');
            header('Location: index.php');
```



```

        exit;
    } else {
        $error = 'Invalid credentials.';
    }
}
}
?>

```

2.4c Sample HTML Code

```

<!-- listing-card.php – included in the browse loop -->
<article class="col-sm-6 col-md-4 col-lg-3 mb-4">
  <div class="card h-100 shadow-sm">
    <figure class="mb-0">
      "
    </figure>
    <div class="card-body d-flex flex-column">
      <h5 class="card-title"><?= htmlspecialchars($listing['title']) ?></h5>
      <p class="text-muted small"><?= htmlspecialchars($listing['category']) ?></p>
      <p class="fw-bold mt-auto">R <?= number_format($listing['price'], 2) ?></p>
      <a href="listing.php?id=<?= $listing['listing_id'] ?>"
        class="btn btn-primary btn-sm">View listing</a>
    </div>
  </div>
</article>

```

2.4d Sample JavaScript Code

Uses the browser's getUserMedia API to access the device camera for QR scanning, then posts the decoded token to the server via AJAX for transaction confirmation.

```

// qr-scan.js – client-side QR scanning with html5-qrcode
document.addEventListener('DOMContentLoaded', () => {
  const scanner = new Html5Qrcode('reader');
  const config = { fps: 10, qrbox: { width: 250, height: 250 } };

  scanner.start(
    { facingMode: 'environment' },
    config,
    (decodedText) => {
      scanner.stop();
      // POST token to server for confirmation
      fetch('confirm_transaction.php', {
        method: 'POST',

```

```

    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ qr_token: decodedText })
  })
  .then(r => r.json())
  .then(data => {
    const msg = document.getElementById('scan-result');
    msg.textContent = data.success
      ? 'Transaction confirmed!'
      : 'Invalid or expired token.';
    msg.className = data.success ? 'alert alert-success' : 'alert alert-danger';
  });
}
);
});

```

2.4e Sample CSS Code

```

/* styles.css – custom theme overrides */
:root {
  --brand-primary: #2E4057;
  --brand-accent: #1B9AAA;
  --card-radius: 12px;
  --transition-fast: 0.2s ease;
}

.card {
  border-radius: var(--card-radius);
  border: none;
  transition: transform var(--transition-fast),
    box-shadow var(--transition-fast);
}

.card:hover {
  transform: translateY(-4px);
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.12);
}

/* Responsive navbar brand */
.navbar-brand { font-weight: 700; color: var(--brand-primary) !important; }

@media (max-width: 576px) {
  .card-img-top { height: 160px; object-fit: cover; }
}

```

```

}
@media (min-width: 992px) {
  .card-img-top { height: 200px; object-fit: cover; }
}

```

2.4f Sample MySQL Table Screenshots

The following screenshots show the key database tables in phpMyAdmin (or equivalent), displaying the table structure and sample data rows to confirm correct implementation.

Table: **users**

Columns:

id	int UN AI PK
name	varchar(100)
email	varchar(150)
password_hash	varchar(255)
phone	varchar(20)
is_admin	tinyint(1)
created_at	datetime

Figure 28: MySQL – USERS table structure

Key	Column	Data
		2 rows matched
2	name, email, pa...	Admin User, admin@vossie.net, \$2y\$12\$q2Ucyd0YHk7WrI1NvSgCOuKi.QrQGNHTDjkTVFq964q1ynEB/FQGa, +27760348254
3	name, email, pa...	Colin Swart, eduv4861087@vossie.net, \$2y\$12\$9NMbFeiHUwh68.MR0J6bGupWMt7UQcP2SoH5wTe4odJZVmEPwEq72, 0760348254

Figure 29: MySQL – USERS table data

Table: **listings**

Columns:

id	int UN AI PK
user_id	int UN
title	varchar(200)
description	text
price	decimal(10,2)
price_type	enum('fixed','wanted')
category	varchar(60)
image	varchar(255)
phone	varchar(20)
status	enum('available','pending','completed')
qr_token	char(64)
created_at	datetime

Object Info Session

Figure 30: MySQL – LISTINGS table

Table: **transactions**

Columns:

id	int UN AI PK
listing_id	int UN
seller_id	int UN
buyer_id	int UN
qr_token	char(64)
confirmed_at	datetime

Figure 32: MySQL – TRANSACTIONS table

2.5 Conclusion

Deliverable 2 presented the design and prototype of Vossie Trading Hall, a student-to-student marketplace for Eduvos users. The prototype demonstrates core functionality which for example includes user registration, listing creation and browsing, and QR-based transaction confirmation, with a responsive interface across devices.

The system design was documented through CRC cards, an EERD, Context Diagram, Data Flow Diagram, Use Case Diagram, and database schema, defining system structure, data flow, and user interactions. An admin module with role-based access was also included to support user and listing management.

The platform is hosted on Microsoft Azure and accessible via the link on the cover page. Overall, this deliverable provides a solid foundation for the final development and implementation phase in Deliverable 3.